

O Gemini disse

Aqui está uma lista de 10 exercícios selecionados para cobrir conceitos fundamentais de algoritmos aplicados aos desafios reais das engenharias Elétrica, Mecânica e Telecomunicações.

⚡ Engenharia Elétrica

1. Cálculo de Queda de Tensão em Corrente Contínua

Contexto: Um engenheiro precisa verificar se a bitola de um cabo é suficiente para alimentar um motor CC a uma certa distância, garantindo que a queda de tensão não ultrapasse 5%.

Pseudocódigo:

Plaintext

Algoritmo QuedaTensao

Var: V_fonte, corrente, resistencia_fio, V_queda, perc_queda: Real

Início

 Leia(V_fonte, corrente, resistencia_fio)

 V_queda <- corrente * resistencia_fio

 perc_queda <- (V_queda / V_fonte) * 100

 Escreva("Queda de tensão:", V_queda, "V")

 Se perc_queda > 5 Então

 Escreva("Alerta: Queda crítica!")

 Senão

 Escreva("Sistema dentro dos padrões.")

 FimSe

Fim

Código Python:

Python

```
v_fonte = float(input("Tensão da fonte (V): "))
```

```
corrente = float(input("Corrente da carga (A): "))
```

```
resistencia = float(input("Resistência total do cabo ( $\Omega$ ): "))
```

```
v_queda = corrente * resistencia
```

```
perc_queda = (v_queda / v_fonte) * 100
```

```
print(f"Queda de Tensão: {v_queda:.2f}V ({perc_queda:.2f}%)")
```

```
if perc_queda > 5:
    print("Resultado: REPROVADO (Acima de 5%)")
else:
    print("Resultado: APROVADO")
```

2. Dimensionamento de Banco de Capacitores

Contexto: Corrigir o fator de potência de uma planta industrial para evitar multas da concessionária. O algoritmo calcula a potência reativa necessária para elevar o fator de potência atual para o desejado.

Pseudocódigo:

Plaintext

Algoritmo CorrecaoFatorPotencia

Var: P_ativa, fp_atual, fp_desejado, Q_necessaria: Real

Início

 Leia(P_ativa, fp_atual, fp_desejado)

 // Usando a relação $Q = P * (\tan(\arccos(fp_atual)) -$

$\tan(\arccos(fp_desejado)))$

$Q_necessaria \leftarrow P_ativa * (\tan(\arccos(fp_atual)) -$

$\tan(\arccos(fp_desejado)))$

 Escreva("Potência do banco de capacitores:", Q_necessaria,
 "kVAR")

Fim

Código Python:

Python

```
import math
```

```
p_ativa = float(input("Potência Ativa (kW): "))
```

```
fp_atual = float(input("Fator de Potência Atual (ex: 0.80): "))
```

```
fp_desejado = float(input("Fator de Potência Desejado (ex: 0.92): "))
```

```
# Cálculo dos ângulos
```

```
phi_atual = math.acos(fp_atual)
```

```
phi_desejado = math.acos(fp_desejado)
```

```
q_necessaria = p_ativa * (math.tan(phi_atual) -  
math.tan(phi_desejado))
```

```
print(f"Capacidade necessária: {q_necessaria:.2f} kVAR")
```

3. Análise de Malha Simples (Lei de Kirchhoff)

Contexto: Determinar a corrente em um circuito série com múltiplas resistências e uma fonte de tensão.

Código Python:

```
Python
tensao = float(input("Tensão da fonte (V): "))
n = int(input("Número de resistores em série: "))
resistores = []

for i in range(n):
    r = float(input(f"Valor do resistor {i+1} ( $\Omega$ ): "))
    resistores.append(r)

r_total = sum(resistores)
corrente = tensao / r_total

print(f"Resistência Equivalente: {r_total}  $\Omega$ ")
print(f"Corrente no circuito: {corrente:.4f} A")
```

Engenharia Mecânica

4. Ciclo de Vida de Fadiga (Critério de Goodman)

Contexto: Estimar se um componente mecânico suportará um carregamento cíclico baseado na tensão média e alternante.

Pseudocódigo:

```
Plaintext
Algoritmo CriterioGoodman
Var: tensao_alt, tensao_med, Se, Sut, seguranca: Real
Início
    Leia(tensao_alt, tensao_med, Se, Sut)
```

```

seguranca <- 1 / ((tensao_alt / Se) + (tensao_med / Sut))
Se seguranca >= 1 Então
    Escreva("Vida infinita estimada. FS:", seguranca)
Senão
    Escreva("Falha por fadiga provável. FS:", seguranca)
FimSe
Fim

```

Código Python:

```

Python
sigma_alt = float(input("Tensão Alternante (MPa): "))
sigma_med = float(input("Tensão Média (MPa): "))
se = float(input("Limite de Resistência à Fadiga - Se (MPa): "))
sut = float(input("Resistência Última à Tração - Sut (MPa): "))

fs = 1 / ((sigma_alt / se) + (sigma_med / sut))

if fs >= 1:
    print(f"Seguro! Fator de Segurança: {fs:.2f}")
else:
    print(f"Risco de Falha! Fator de Segurança: {fs:.2f}")

```

5. Conversão de Torque e Potência em Motores

Contexto: Calcular o torque disponível no eixo de um motor de combustão interna ou elétrico a partir da potência e rotação (RPM).

Código Python:

```

Python
import math

pot_cv = float(input("Potência do motor (CV): "))
rpm = float(input("Rotação (RPM): "))

# Conversão CV para Watts (1 CV ≈ 735.5 W)
pot_w = pot_cv * 735.5
# Velocidade angular (rad/s)
omega = (2 * math.pi * rpm) / 60

torque = pot_w / omega

```

```
print(f"Torque resultante: {torque:.2f} N.m")
```

6. Dilatação Térmica de Tubulações

Contexto: Calcular a variação no comprimento de uma tubulação de aço inox que transporta vapor superaquecido para prever a necessidade de juntas de expansão.

Código Python:

```
Python
l_inicial = float(input("Comprimento inicial (m): "))
temp_inicial = float(input("Temperatura inicial (°C): "))
temp_final = float(input("Temperatura de operação (°C): "))
# Coeficiente para aço inox aprox 17.3 x 10^-6
alpha = 17.3e-6

delta_t = temp_final - temp_inicial
delta_l = l_inicial * alpha * delta_t

print(f"A tubulação irá expandir: {delta_l*1000:.2f} mm")
```

Engenharia de Telecomunicações

7. Cálculo de Orçamento de Link (Link Budget)

Contexto: Determinar se a potência de sinal recebida em uma antena receptora é suficiente para manter a comunicação, considerando perdas no espaço livre.

Pseudocódigo:

```
Plaintext
Algoritmo LinkBudget
Var: Pt, Gt, Gr, L, Pr: Real
Início
    Leia(Pt_dbm, Gt_dbi, Gr_dbi, Perdas_db)
    Pr <- Pt_dbm + Gt_dbi + Gr_dbi - Perdas_db
    Escreva("Potência Recebida:", Pr, "dBm")
    Se Pr < -90 Então
```

```
        Escreva("Sinal insuficiente (abaixo da sensibilidade)")
    FimSe
Fim
```

Código Python:

```
Python
pt_dbm = float(input("Potência de Transmissão (dBm): "))
gt_dbi = float(input("Ganho Antena TX (dBi): "))
gr_dbi = float(input("Ganho Antena RX (dBi): "))
distancia_km = float(input("Distância (km): "))
frequencia_mhz = float(input("Frequência (MHz): "))

# Perda de Percurso no Espaço Livre (FSPL)
fspl = 32.44 + 20*math.log10(distancia_km) +
20*math.log10(frequencia_mhz)
pr = pt_dbm + gt_dbi + gr_dbi - fspl

print(f"Perda no espaço livre: {fspl:.2f} dB")
print(f"Potência Recebida Estimada: {pr:.2f} dBm")
```

8. Conversão Binário para Sinal Unipolar (Codificação de Linha)

Contexto: Simular como uma sequência de bits é transformada em níveis de tensão para transmissão em um cabo.

Código Python:

```
Python
bits = input("Digite a sequência de bits (ex: 1011): ")
v_high = 5.0
v_low = 0.0

print("Sinal de saída (V):")
for bit in bits:
    if bit == '1':
        print(f"| {v_high}V ", end="")
    else:
        print(f"| {v_low}V ", end="")
print("|")
```

9. Cálculo de Capacidade de Canal (Teorema de Shannon)

Contexto: Calcular a taxa máxima teórica de transmissão de dados em um canal com ruído.

Código Python:

```
Python
import math

bw = float(input("Largura de Banda (Hz): "))
snr_db = float(input("Relação Sinal-Ruído - SNR (dB): "))

# Converter SNR de dB para linear
snr_linear = 10**(snr_db / 10)

capacidade = bw * math.log2(1 + snr_linear)

print(f"Capacidade máxima do canal: {capacidade/1e6:.2f} Mbps")
```

Exercício Interdisciplinar

10. Monitoramento de Sensores com Média Móvel

Contexto: Em qualquer engenharia, sensores de leitura (temperatura, pressão, tensão) sofrem com ruídos. Este algoritmo suaviza a leitura usando uma média móvel de 3 amostras.

Código Python:

```
Python
leituras = []
print("Simulação de Sensor (digite 0 para parar)")

while True:
    valor = float(input("Valor lido: "))
    if valor == 0: break

    leituras.append(valor)

    if len(leituras) >= 3:
        # Pega as últimas 3 amostras
        janela = leituras[-3:]
```

```
media = sum(janela) / 3
print(f"Média Móvel (Suavizada): {media:.2f}")
else:
    print("Aguardando mais amostras...")
```